Python Programming Lab. [Batch-B3]

Assignment Submission Details

Field	Details
Student Name *	Dilip Balu Magar
Student PRN *	125M1H055
Course Name	Python Programming Lab. (MCA31PC06)
Academic Year	2025-26 (Semester-1)
Course Teacher	Prof. Prakash Ukhalkar
Assignment Name	Assignments based on Functions in Python 2.
Assignment Number	Assignment 05
Submission Date *	18-11-2025

1.Create a Python function that calculates the area of a rectangle. The function should have default arguments for the width and height. Test the function with and without providing the arguments.

In [ ]:
```
def cal_area_rec(length=10, breadth=5):
    print("length:",length)
    print("breadth",breadth)
    return length * breadth

l=int(input("enter lenght of rectangle"))
b =int(input("enter breadth of rectangle"))

print(f"Calculating area by providing values ")
print("area:",cal_area_rec(l,b))
print(f"Calculating area without providing any values ")
print("area:",cal_area_rec())
```

```
Calculating area by providing values
length: 5
breadth 6
area: 30
Calculating area without providing any values
length: 10
breadth 5
area 50
```

2.Develop a Python function that takes a list of numbers and returns the minimum, maximum, and average of the list. Call the function and print the results..

In [11]:
```
def cal(numbers):

    min_value = min(numbers)
    max_value = max(numbers)
    total = sum(numbers)
    avg = total / len(numbers)

    return (min_value, max_value, avg)


numbers = [1, 2, 3, 4, 5, 6, 6, 7, 78]

min_val, max_val, avg_val = cal(numbers)

print(f"Given List: {numbers}")

print(f"Minimum Value: {min_val}")
print(f"Maximum Value: {max_val}")
print(f"Average Value: {avg_val:.2f}")
```

```
Given List: [1, 2, 3, 4, 5, 6, 6, 7, 78]
Minimum Value: 1
Maximum Value: 78
Average Value: 12.44
```

3.Create a Python function that accepts any number of arguments and returns their product. Test the function with different numbers of arguments..

In [12]:
```
def calculate_product(*args):

    result = 1

    for number in args:
        result *= number

    return result


product_two = calculate_product(5, 10)
print(f"Product of 5 and 10: {product_two}")


product_multiple = calculate_product(2, 3, 4, 0.5)
print(f"Product of 2, 3, 4, and 0.5: {product_multiple}")
```

```
Product of 5 and 10: 50
Product of 2, 3, 4, and 0.5: 12.0
```

4.Write a Python program that uses a lambda function to sort a list of tuples based on the second element of each tuple. Test the sorting function with sample data. .

In [13]:
```
data = [
    ('apple', 50),
    ('banana', 10),
    ('cherry', 80),
    ('orange', 20),
    ('mango', 50),
    ('grapes', 50)
]

print("Given List")
for item in data:
    print(item)

# x represents a single tuple (e.g., ('apple', 50)).
# x[1] extracts the element at index 1 (50).
# The sorting will be based on numerical value.

sorted_data = sorted(data, key=lambda x: x[1])

print("\n Sorted List by second element in list of tuple")
for item in sorted_data:
    print(item)
```

```
Given List
('apple', 50)
('banana', 10)
('cherry', 80)
('orange', 20)
('mango', 50)
('grapes', 50)

 Sorted List by second element in list of tuple
('banana', 10)
('orange', 20)
('apple', 50)
('mango', 50)
('grapes', 50)
('cherry', 80)
```

5.Develop a Python function that calculates the factorial of a number using recursion. Test the function with various inputs.

In [14]:
```
def fact(n):

    if n < 0:
        return "Factorial is not defined for negative numbers."

    if n == 0 or n == 1:
        return 1
    else:
        return n * fact(n - 1)


num1 = 5
result1 = fact(num1)
```

```
print(f"Factorial of {num1}: {result1}")


num2 = 7
result2 = fact(num2)
print(f"Factorial of {num2}: {result2}")

num3 = 0
result3 = fact(num3)
print(f"Factorial of {num3}: {result3}")
```

```
Factorial of 5: 120
Factorial of 7: 5040
Factorial of 0: 1
```

6.Write a Python function that takes a list of numbers as an argument and returns a new list with each element squared. Call the function and print the new list.

In [15]:
```
def square(input_list):


    squared_list = [number ** 2 for number in input_list]

    return squared_list


original_list = [2, 3, 5, 8, -4, 0.5]

print(f"Original List: {original_list}")

new_squared_list = square(original_list)


print(f"Squared List: {new_squared_list}")
```

```
Original List: [2, 3, 5, 8, -4, 0.5]
Squared List: [4, 9, 25, 64, 16, 0.25]
```

7.Create a Python function that takes a list of strings and converts each string to uppercase. The function should modify the original list. Test the function with a sample list.

In [16]:
```
def convert_to_uppercase(string_list):

    for i in range(len(string_list)):
        string_list[i] = string_list[i].upper()

words = ["hello", "dilip", "magar"]


print(f"List before converting to upper case: {words}")

convert_to_uppercase(words)
```

```
print(f"List after converting to upper case: {words}")
```

```
List before converting to upper case: ['hello', 'dilip', 'magar']
List after converting to upper case: ['HELLO', 'DILIP', 'MAGAR']
```

8.Develop a Python function that checks whether a given number is prime. The function should return True if the number is prime and False otherwise

In [19]:
```
import math

def is_prime(n):

    if n <= 1:
        return False

    if n == 2:
        return True

    if n % 2 == 0:
        return False


    sqrt = int(math.sqrt(n))
    for i in range(3, sqrt + 1, 2):
        if n % i == 0:
            return False

    return True

n=int(input("enter a number to check prime or not"))
result = is_prime(n)
print(f"{n} is prime:{result}")
```

```
4 is prime:False
```

9.Write a Python function that takes a dictionary as an argument and prints each key-value pair in the format key: value. Test the function with a sample dictionary.

In [20]:
```
def print_dictionary(data_dict):

    for key, value in data_dict.items():
        print(f"{key}: {value}")

student_data = {
    1: "dilip",
    2: "tushar",
    3: "harsh",
    4: "harry",
    5: "sudhir"
}

print("Given dictionary:")
print_dictionary(student_data)
```

```
Given dictionary:
1: dilip
2: tushar
3: harsh
4: harry
5: sudhir
```

10.Create a Python function that contains another function inside it. The inner function should perform a calculation, and the outer function should return the result of the inner function. Demonstrate the use of the nested function.

In [21]:
```python
def cal_sum(a, b):

    def addition(x, y):
        return x + y

    result = addition(a, b)
    return result


num1 = 15
num2 = 7
sum = cal_sum(num1, num2)

print(f"Numbers to add: {num1} and {num2}")
print(f"Result from nested function: {sum}")
```

```
Numbers to add: 15 and 7
Result from nested function: 22
```